

Západočeská univerzita v Plzni
Fakulta aplikovaných věd

Programování v jazyce C

Vizualizace grafu matematické funkce

Daniel Mašek

Obsah

1	Zadání	1
1.1	Specifikace vstupu programu	1
1.2	Způsob zápisu funkcí	2
1.3	Specifikace rozsahu zobrazení	2
1.4	Matematické funkce v zápisu	3
1.5	Specifikace výstupu programu	3
2	Analýza úlohy	4
3	Popis implementace	7
3.1	Použité struktury	7
3.2	Konstanty	8
3.3	Validace vstupů	9
3.4	Dělení vstupních dat	9
3.5	Konverze do postfixové notace	9
3.6	Vytvoření souboru	10
4	Uživatelská příručka	11
5	Závěr	12

1 Zadání

Naprogramujte v ANSI C přenositelnou konzolovou aplikaci, která jako vstup načte z parametru na příkazové řádce matematickou funkci ve tvaru $y = f(x)$; $x, y \in R$, provede její analýzu a vytvoří soubor ve formátu PostScript s grafem této funkce na zvoleném definičním oboru.

Program se bude spouštět příkazem **graph.exe** **<func>** **<out-file>** [**<limits>**]. Symbol **<func>** zastupuje zápis matematické funkce jedné reálné nezávislé proměnné (funkce ve více dimenzích program řešit nebude, nalezne-li během analýzy zápisu funkce více nezávisle proměnných než jednu, vypíše srozumitelné chybové hlášení a skončí). Závisle proměnná (zde y) je implicitní, a proto se levá strana rovnosti v zápisu nebude uvádět. Symbol **<out-file>** zastupuje jméno výstupního postscriptového souboru.

Výsledkem práce programu bude soubor ve formátu PostScript, který bude zobrazovat graf zadané matematické funkce s vyznačenými souřadnými osami a (aspoň) význačnými hodnotami definičního oboru a oboru hodnot funkce.

Pokud nebudou na příkazové řádce uvedeny alespoň dva argumenty, vypíše chybové hlášení a stručný návod k použití programu v angličtině podle běžných zvyklostí.

1.1 Specifikace vstupu programu

Vstupem programu jsou pouze parametry na příkazové řádce: Prvním (a nejdůležitějším) parametrem je matematická funkce, kterou má program zpracovat. S ohledem na to, že může její zápis obsahovat mezery, napište program tak, aby akceptoval jak zápis funkce obklopený uvozovkami, tak bez nich, tj. aby bylo možné program spouštět jak zadáním příkazu **graph.exe** **x+x²**, tak příkazem **graph.exe** **"x + x²"**. Také použití mezer v zápisu funkce musí být libovolné. Je také možné, že uživatel vašeho programu z nepořádnosti někde v zápisu mezery použije a někde zase ne, třeba takto: **"sin (x ²)*cos(x / 2.0)"** – tato varianta zápisu je také akceptovatelná.

1.2 Způsob zápisu funkcí

V zápisu matematické funkce, jejíž graf bude program generovat, se mohou vyskytnout tyto symboly: konstanty, proměnná, aritmetické operátory, funkce, závorky. Jiné symboly zápis obsahovat nesmí – pokud se v zápisu objeví, měl by program reagovat srozumitelným chybovým hlášením.

Konstanty (mohou být ve všech akceptovatelných formátech zápisu celého či reálného čísla v jazyce C, tj. např. i **.5E-02** či **1E0**). S ohledem na to, že má program řešit jen 2D grafy, jediná možná nezávislá proměnná, která se může v zápisu funkce vyskytovat je x .

následující tabulka ukazuje veškeré povolené matematické operace:

Operace	Zápis operátoru	Příklad
unární mínus	$-$	$-x$
sčítání	$+$	$x + 2$
odčítání	$-$	$x - 2$
násobení	$*$	$2 * x$
dělení	$/$	$x/2$
umocnění	$^$	x^2

Tabulka 1: Povolené matematické operace

Jiné operátory nejsou povolené. Priorita operátorů je dána matematickými pravidly, případně upravena závorkami ($()$) – jiné druhy závorek se v zápisu funkce nesmí vyskytovat.

1.3 Specifikace rozsahu zobrazení

Třetí – **nepovinný** – parametr $[\langle \text{limits} \rangle]$ na příkazové řádce slouží k předání informací o rozsazích zobrazení grafu analyzované funkce, tedy o horní a dolní mezi definičního oboru a oboru hodnot funkce. Má tvar:

$$\langle \text{xdol} \rangle : \langle \text{xhor} \rangle : \langle \text{ydol} \rangle : \langle \text{yhor} \rangle$$

Například příklad pro spuštění se zadanými rozsahy může vypadat takto: **graph.exe "sin(x^2)*cos(x)" sin2cos.ps 6.28:12.56:-1:1**. Není-li rozsah uveden (protože se jedná o nepovinný parametr), použijte implicitní nastavení, které nechť je pro definiční obor i obor hodnot $\langle -10; 10 \rangle$.

1.4 Matematické funkce v zápisu

Váš program by měl akceptovat v zápisu zobrazované funkce některé běžně používané matematické funkce. Funkce uvedené v následujícím výčtu program musí akceptovat, jinak nebude uznán za řádně dokončený: absolutní hodnota **abs**; funkce e^x **exp**, přirozený logaritmus **ln**, dekadický logaritmus **log**; goniometrické funkce **sin**, **cos**, **tan**; cyklometrické funkce **asin**, **acos**, **atan**; hyperbolometrické funkce **sinh**, **cosh**, **tanh**.

1.5 Specifikace výstupu programu

Výstup programu bude směřován na konzoli pouze v případě chyby v zadání parametrů. Pokud budou všechny parametry akceptovatelné a funkce k zobrazení syntakticky správně zapsaná, nemusí program vypisovat na konzoli nic. Pokud program neobdrží požadovaný počet parametrů na příkazovém řádku či pokud je funkce špatně zapsána, měl by vypsat srozumitelné chybové hlášení v angličtině a skončit nenulovým návratovým kódem.

Popis chybového stavu	Návratová hodnota funkce <code>int main()</code>
Bez chyby/korektní ukončení činnosti programu.	0
Nebyly předány správné parametry na příkazové řádce při spuštění programu.	1
Řetězec předaný programu jako první parametr není akceptovatelným zápisem matematické funkce.	2
Řetězec předaný programu jako druhý parametr nemůže být v hostitelském operačním systému názvem souboru nebo takový soubor nelze vytvořit či do něj ukládat informace.	3
Uvedený nepovinný třetí parametr nelze dekodovat jako rozsahy zobrazení.	4

Tabulka 2: Chybové stavy a návratové hodnoty funkce

Výsledkem činnosti programu by měl být soubor ve formátu PostScript, který bude obsahovat vykreslený graf zadané funkce.

2 Analýza úlohy

Dle požadavků je nejdříve vhodné zkontrolovat, jestli byly předány argumenty a popřípadě zkontrolovat, jestli jsou validní a následně zkontrolovat správnost vstupu.

Prvním argumentem je funkce, tedy je potřeba udělat syntaktickou analýzu. Jedním ze způsobu realizace je analýza rekurzivním sestupem, já jsem se rozhodl vymyslet si vlastní postup. Můj postup při řešení problému spočívá v tom, že při každém znaku určím, jestli na dané pozici může být tento znak a pokud se jedná o znak, za kterým může následovat další znak, který k němu patří tak čtu celou sadu znaků dokud splňují podmínku a zkontroluji správnost celku. Např. když se jedná o vícemístné číslo tak přečtu celé číslo a zjistím, jestli je správně zapsané. Nevýhodou mého řešení je, že je zdlouhavé, ovšem přijde mi jednodušší na pochopení než rekurzivní přístup. Co se časové složitosti týče, tak jsou srovnatelné.

Dalším problémem je druhý argument, tedy soubor do kterého se má uložit výsledek programu. Jelikož je PostScript podporován na všech platformách, tak není třeba kontrolovat, jestli jej lze na daném systému vytvořit. Je potřeba pouze kontrola správnosti názvu souboru, tedy koncovky, která musí být .ps.

Poslední argument je nepovinný, je tedy potřeba zjistit, jestli byl předán. Pokud předán nebyl, musí se použít pro definiční obor i obor hodnot limity od -10 do 10 . Pokud předán byl, tak se musí zkontrolovat jestli je správný a uložit ho. To lze udělat kontrolou buď celého vstupu a nebo ho rozdělit a kontrolovat po částech. Rozhodl jsem se ho kontrolovat po částech, protože je třeba limity rozdělit v obou případech a tímto způsobem bude implementace kontrolující funkce jednodušší. Nevýhoda je ovšem ta, že když bude poslední hranice nesprávná, tak se kontroly a ukládání prováděly zbytečně.

Po načtení argumentů je potřeba začít zpracovávat funkci. Jelikož je třeba převést funkci na reverzní polskou notaci, tak je vhodné funkci tokenizovat na funkce, konstanty a proměnné a ostatní. Funkci **strtok** nelze použít, protože potřebuje oddělovač. Je tedy potřeba funkce, která funkci tokenizuje. pro jednodušší práci by bylo vhodné každý token označit, jestli jeho hodnota je funkce, konstanta nebo něco jiného. tokenizační funkce může fungovat na podobném principu jako validace funkce, tedy rekurzivním sestupem a nebo mým postupem řešení. Rozhodl jsem se pro můj postup, protože je již

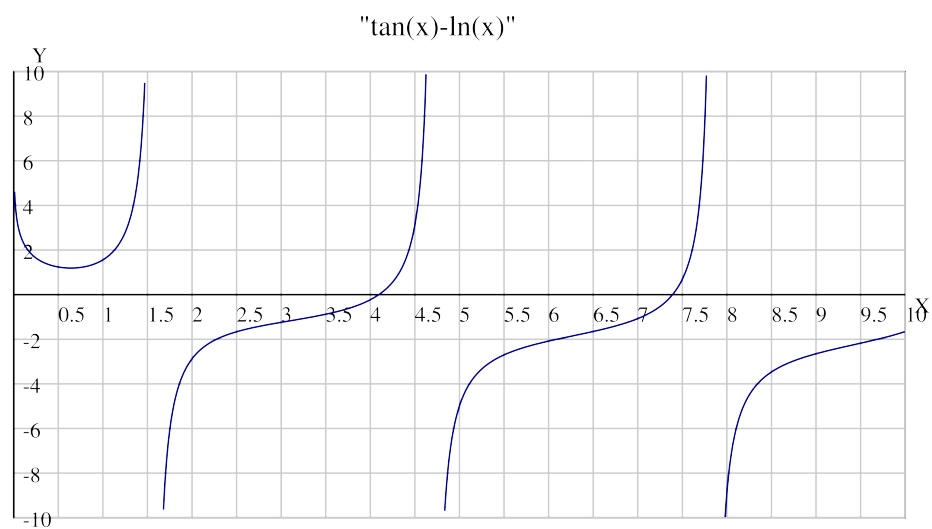
naimplementovaný a je potřeba ho pouze trochu upravit. Jelikož je funkce již validována, tak jí stačí pouze převést na tokeny. výhodou mého přístupu je, že nebudu muset zjišťovat co token obsahuje a bude pouze stačit přechíst označení tokenu a podle něj určit co se s tokenem má stát.

pro převod do reverzní polské notace jsem použil algoritmus shunting yard. Funkci je vhodné převést kvůli tomu, aby bylo její vyhodnocování jednodušší a to díky její vlastnosti, že operátory jsou seřazeny podle jejich pořadí vyhodnocování. také lze přímo do PostScriptu zapsat funkci v reverzní polské notaci a obstarávat vykreslení přímo v souboru. Kromě shunting yard by šel použít i abstraktní syntaktický strom, ovšem ten je značně složitější na implementaci. Na druhou stranu je ale snadno rozšiřitelný. Ještě by šel požit přístup pomocí 2 zásobníků, ovšem ten je neintuitivní a paměťově náročnější než shunting yard.

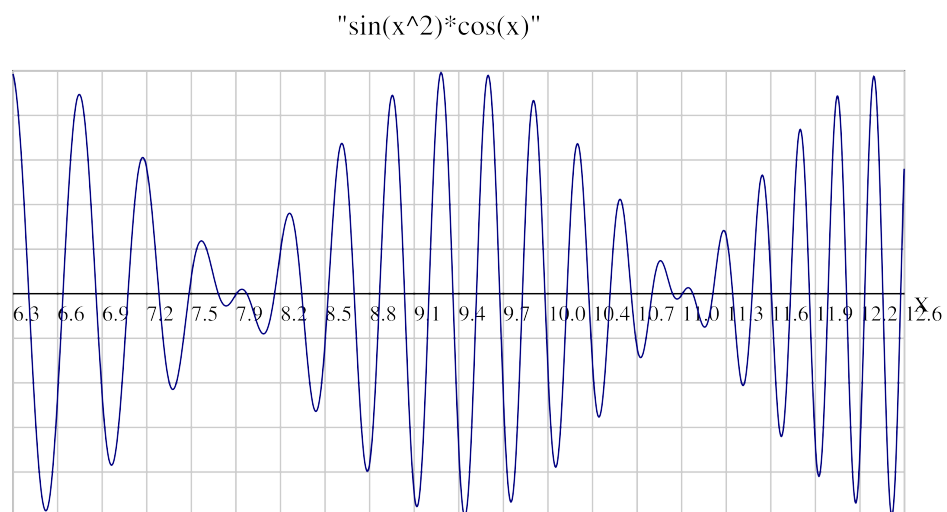
Posledním problémem je vytvoření PostScript souboru. V něm potřebuji vykreslit funkci s osami x a y a číselnými řadami v rozsahu limit. Aby měl graf rozumnou velikost, tak jsem se rozhodl, pro poměr stran 2:1 a šířkou grafu na šířku papíru A4. abych neutralizoval možnost kreslení mimo plátno, tak jsem se rozhodl udělat odsazení od krajů. Pozici os jsem se rozhodl udělat závislou na zobrazovaných intervalech definičního oboru a oboru hodnot, tedy pozice os není pevně stanovena. pro vykreslení funkce jsem existuje několik způsobů. prvním způsobem je zapsat do souboru funkci v reverzní polské notaci a nechat jí vytvořit PostScriptem. Tento způsob má ale velkou nevýhodu v tom, že PostScript nepodporuje všechny požadované matematické funkce a tedy je nutné nepodporované funkce vyjádřit jiným způsobem. Například hyperbolický sinus by musel být přepsán jako:

$$\sinh(x) = \frac{e^x - e^{-x}}{2} \quad (1)$$

Dalším problémem jsou mocniny, protože x^2 je v reverzní polské notaci " x 2 ^" a PostScript očekává " 2 x ^". Z těchto důvodů jsem se rozhodl pro způsob vyčíslování funkce a zapisování čar do souboru manuálně. Tam je ovšem problém s nespojitými funkcemi a je nutné si stanovit začátek nové čáry, když dojde k bodu nespojitosti. Tento způsob také vytváří násobně větší soubory kvůli tomu, že všechno je zapsáno v souboru a není to generováno PostScriptem. Rozhodl jsem se pro něj kvůli jednoduchosti implementace, protože jeho nedostatky nejsou příliš závažné a pro řešení této úlohy je postačující.



Obrázek 1: Ukázka výstupu programu pro funkci $\tan(x) - \ln(x)$



Obrázek 2: Ukázka výstupu programu pro funkci $\sin(x^2) \cdot \cos(x)$

3 Popis implementace

Program jsem implementoval dle analýzy, tedy nejdřív proběhne kontrola parametrů příkazové řádky jestli byly předány povinné parametry, pokud ne, tak program skončí chybovou hláškou dle zadání. Následně se provede kontrola předání třetího nepovinného parametru. Pokud předán byl, tak se uloží a program pokračuje, pokud ne tak se uloží nula. Pak probíhá validace funkce. Pokud funkce není validní, tak program skončí chybovým kódem dle zadání. Dále se provede kontrola výstupního souboru, jestli je ve správném formátu a pokud není, tak program skončí chybovou hláškou. Nyní se provede Dělení limit podle dříve uložené hodnoty a vyhodnotí se. Pokud předán nebyl, použijí se limity "-10:10:-10:10", pokud ano, tak se zkontrolují a uloží.

Po kontrole vstupů se alokuje paměť pro tokeny v infixové a postfixové notaci a funkce se rozdělí na jednotlivé tokeny představující funkce, operátory a jiné povolené symboly. Jelikož byla funkce již dříve kontrolována, tak se neprovádí kontrola teď. Následně se funkce převede do postfixové notace, kvůli jednoduššímu vyčíslování při zápisu do souboru. Po převodu do postfixu se uvolní paměť použitá na infixovou notaci a začne se generovat soubor. Po úspěšném vytvoření souboru se program ukončí. Při neúspěchu skončí s chybovým kódem 5.

3.1 Použité struktury

```
1 typedef enum
2 {
3     NUM,    /*number*/
4     VAR,    /*variable x*/
5     OP,     /*operator (+,-,* etc.)*/
6     FUNC,   /*function (sin,cos etc.)*/
7     LBR,    /*left bracket '(' */
8     RBR,    /*right bracket ')' */
9     END     /*end of token*/
10 } token_type;
```

Tato struktura slouží k označení tokenu typem, tedy pokud se rozhodne že token je číslo, tak bude mít typ **NUM**. Tento přístup usnadní implementaci algoritmu shunting yard.

```

1 typedef struct
2 {
3     token_type type;          /*token type*/
4     char value[MAX_TOK_LEN]; /*token*/
5 } token;

```

Tato struktura představuje token, tedy matematickou funkci, operátor a ostatní možné řetězce. Atribut **type** představuje typ tokenu popsany výše a atribut **value** je hodnota tokenu typu pole symbolů.

```

1 typedef struct
2 {
3     double x_start, x_end, y_start, y_end;
4 } graph_limits;

```

Struktura **graph_limits** představuje rozsah hodnot na výsledném grafu a využívá se při výpočtech potřebných u vykreslení.

3.2 Konstanty

```

1 #ifndef CONSTS_H
2 #define CONSTS_H
3
4 #define MAX_TOK_LEN 64
5 #define NUM_OF_SAMPLES 8000
6 #define DEFAULT_RANGE_X 20
7 #define DEFAULT_RANGE_Y 10
8 #define MARGIN 50
9 #define DISCONTUITY_THRESHOLD_SCALE 0.75
10 #endif

```

Všechny konstanty jsou uloženy v jednom hlavičkovém souboru **consts.h** kvůli přehlednosti kódu.

Konstanta **MAX_TOK_LEN** představuje maximální délku tokenu. Velikost je předimenzovaná hlavně kvůli číslům, které mohou mít velkou délku. Pokud velikost čísla přeteče tak to program zjistí a vykreslí prázdný graf.

DEFAULT_RANGE_X Představuje počet čísel na ose X.

DEFAULT_RANGE_Y představuje počet čísel na ose Y.

Konstanta **MARGIN** představuje odsazení grafu od kraje stránky v bodech.

Konstanta **DISCONTUITY_THRESHOLD_SCALE** slouží k vynáso-
bení rozsahu Y a pokud hodnota funkce skočí o tuto hodnotu, tak se určí, že
je tam bod nespojitosti.

3.3 Validace vstupů

Veškerou validaci v programu obstarává modul **validator.h**.

Kontrola funkce spočívá v nastavených pravidlech, to znamená že každý
znak se kontroluje jestli je číslo, písmeno, operátor nebo závorka. každé z
pravidel má svá podpravidla, například že před otevírací závorkou může být
nic, operátor nebo funkce a pokud je tam něco jiného, výraz se vyhodnotí
jako nesprávný. Takováto pravidla jsou nadefinovaná pro každý možný vstup.
Čísla a funkce jsou řešena načtením do pomocného pole a jsou validována
vcelku. čísla mají složitá pravidla, protože mohou obsahovat i jiné znaky než
čísllice.

Validace výstupního souboru probíhá pouze kontrolou koncovky souboru,
protože PostScript je podporován na všech platformách.

Kontrola limit probíhá trochu jinak. Nejdřív jsou rozděleny na čísla
a každé číslo je kontrolováno podle stejných pravidel jako číslo ve funkci.

3.4 Dělení vstupních dat

veškerý převod na tokeny obstarává modul **parser.h**

tokenizace funkce probíhá obdobně jako její kontrola s tím rozdílem, že
se nekontroluje syntaktická správnost a vytvořené tokeny se ukládají do pole
vytvořeného při startu programu.

dělení limit využívá funkci **strtok** a dělí se podle dělicího znaku, každý
vytvořený token se poté validuje.

3.5 Konverze do postfixové notace

Konverzi do postfixové notace obstarává **converter.h**, který využívá
stack.h Konverze probíhá pomocí shunting-yard algoritmu. Ten pracuje tak
že čte tokeny od začátku do konce. Pokaždé co přijde číslo nebo proměnná,

tak je položena na vrchol zásobníku a u operátorů jako jsou funkce a matematické operace se vyndají ze zásobníku všechny operátory vloží se aktuální operátor a pak všechny operátory, kteří byli vyjmuti ze zásobníku.

3.6 Vytvoření souboru

Pro vytváření souborů typu PostScript je určen modul **postscript_creator.h** a pro vyčíslování funkce se používá modul **evaluator.h**

Při vytváření souboru napřed vytvořím soubor, nebo přepíšu soubor se stejným názvem, který již existuje a napíšu hlavu souboru.

```
1 double graph_width = 595;
2 double graph_height = (graph_width+2*MARGIN)/2; // this is
  to make the graph 2:1 after subtracting the margins
3 double x_interval = (limits.x_end - limits.x_start) /
  DEFAULT_RANGE_X;
4 double y_interval = (limits.y_end - limits.y_start) /
  DEFAULT_RANGE_Y;
5
6 // Write PostScript header
7 fprintf(file, "%!PS-Adobe-3.0\n");
8 fprintf(file, "%%BoundingBox: 0 0 %lf %lf\n",
  graph_width, graph_height);
9 fprintf(file, "%%Title: \n", function);
10 fprintf(file, "%%Creator: Daniel Masek\n");
```

Poté vytvořím plátno, tedy ohraničení a kříž. pozice kříže není pevná, záleží na definičním oboru. Dále se vykreslí název grafu, což je funkce v infixovém tvaru.

Nakonec se vykreslí samotná funkce, která se vyčísluje, zkontroluje se, jestli je výsledek číslo a jestli je v limitách a jestli nedošlo ke skoku, což znamená bod nespojitosti. Pokud se jedná o bod nespojitosti, tak se přeruší čára a začne nová v posledním bodě. Pak se překonvertují souřadnice, a vykreslí se čára po čáře. Dokud se nedojde na konec definičního oboru a zavře se soubor. Poté se program ukončí s kódem 0.

4 Uživatelská příručka

Pro spuštění programu je potřeba mít nainstalovanou sadu kompilátorů gcc a to jak pro Linux, tak i pro Windows. Na Linuxu je třeba pro zkompilování zavolat příkaz **"make"** nebo **"make <filename>"**, kde <filename> je název makefile k použití. Pro Windows je to obdobné s rozdílem že místo příkazu **"make"** se použije **"mingw32-make"**.

Nyní, když je program zkompilovaný, tak je možn ho spustit. Na Windows je potřeba otevřít konzoli nebo PowerShell a dostat se do adresáře se zkompilovaným souborem. Pro zpuštění v konzoli se zadá příkaz:

```
C:\> graph.exe < function >< filename>[< limits>]
```

Kde <function> je matematická funkce, <filename> je název souboru (například test.ps) a <limits> je nepovinný parametr limit. (ro PowerShell to je obdobné akorát je třeba na začátku přidat **"/"**). Pro Linux je spouštění souboru stejné jako u PowerShellu, tedy:

```
C:\> ./graph.exe < function >< filename>[< limits>]
```

Po zadání spouštěcího příkazu se program rozběhne a pokud nevypíše chybové hlášení a skončil s kódem 0, tak vše proběhlo v pořádku a soubor byl vytvořen. Pokud ale byla v zadaných parametrech chyba, tak program vypíše co je za problém a skončí s chybovým kódem dle zadání. Výsledky je možné vidět na stránce 6

Zde je ukázka správně zapsaného Spouštěcího příkazu:

```
C:\> ./graph.exe sin(x^2)*cos(x) test.ps 6.28:12.56:-1:1
```

5 Závěr

Výsledkem mé semestrální práce je program vykreslující graf zadané funkce který je časově nenáročný a v běžných případech je jeho běh ukončen úspěchem za méně než 1 vteřinu. Ovšem výsledný graf není perfektní z důvodu zvoleného přístupu k vykreslení grafu, a to v ohledu na velmi rychle rostoucí funkce, kde se občas stane, že graf není dokreslen až k okrajům jako na obrázku 1.

Zadání jsem splnil v plném rozsahu. Bylo by ovšem vhodné upravit zápis do souboru a validační funkce, které by bylo možné značně zjednodušit.

Během řešení této úlohy jsem narazil na problémy během vykreslování funkcí. Zejména nespojité funkce a funkce definované pouze na určitém definičním oboru. Na začátku řešení jsem měl také problém s převáděním funkce do tokenů, a to protože jsem špatně použil ukazatel.